

EASYMODE: Complete Technical Tutorial & Viva Survival Guide

The Ultimate Preparation Guide for Tomorrow Morning's Defense *Read this guide with your teammates. By the time you finish, you will know exactly how the whole project works and how to answer any question the professors throw at you.*

Table of Contents

1. [The 60-Second Elevator Pitch \(Memorize This\)](#)
2. [What Does EasyMode Actually Do? \(In Plain English\)](#)
3. [The High-Level Architecture \(The 3 Main Parts\)](#)
4. [Step-by-Step: What Happens When You Click "Analyze Reviews"](#)
5. [The 4 "Secret Weapons" \(Engineering Tricks Professors Love\)](#)
6. [Codebase Walkthrough: What Every File Does](#)
7. [Why UN Sustainable Development Goals \(SDGs\)?](#)
8. [Top 20 Brutal Viva Questions & Exact Answers to Say](#)
9. [Tomorrow Morning's Demo Checklist & Secret Commands](#)

1. The 60-Second Elevator Pitch (Memorize This)

*"Good morning, Professors. Our project is **EasyMode**, a privacy-first, on-device AI browser extension that synthesizes e-commerce product reviews into instant, honest purchasing advice.*

Today, shoppers face thousands of reviews per product, and over 80% are uninformative 5-star ratings that hide serious defects. Commercial tools like Amazon Rufus have a conflict of interest because they want you to buy, so they rarely say 'Pass'. Cloud extensions like Fakespot track your browsing data and cost money per query.

*EasyMode solves this by running a compact, quantized AI model—**Meta's Llama 3.2 1B**—directly on the user's laptop using Ollama. It extracts reviews in under 0.4 seconds, intelligently filters out boilerplate noise, guarantees representation for critical 1-star complaints, and outputs a clear verdict: **Strong Buy**, **Mixed**, or **Pass** with concrete pros and cons in just 2.5 seconds—with zero cloud fees, zero data tracking, and 100% offline capability."*

2. What Does EasyMode Actually Do? (In Plain English)

Imagine you want to buy a water gel moisturizer or wireless earbuds on Amazon.

- The listing says **4.3 out of 5 stars** with **15,000 ratings**.
- You can't read 50 pages of reviews.
- If you look at the top page, it's mostly people saying *"Nice product"*, *"Fast delivery"*, or *"Best moisturizer ever"*.
- But buried on page 3 or 4, ten people are complaining: *"Gave me a severe allergic reaction and bad skin redness"*, or *"Jar came cracked and unsealed"*.

EasyMode fixes this in 2.5 seconds:

1. You click the extension icon on the Amazon page.
2. It quickly grabs recent customer reviews.
3. It cleans out the junk (like *"Verified Purchase"*, sizing info, and 1-word comments like *"good"*).
4. It picks a balanced mix (not just good reviews, but purposely selecting negative reviews so defects are visible).
5. It hands this balanced mix to a local AI running on your laptop.

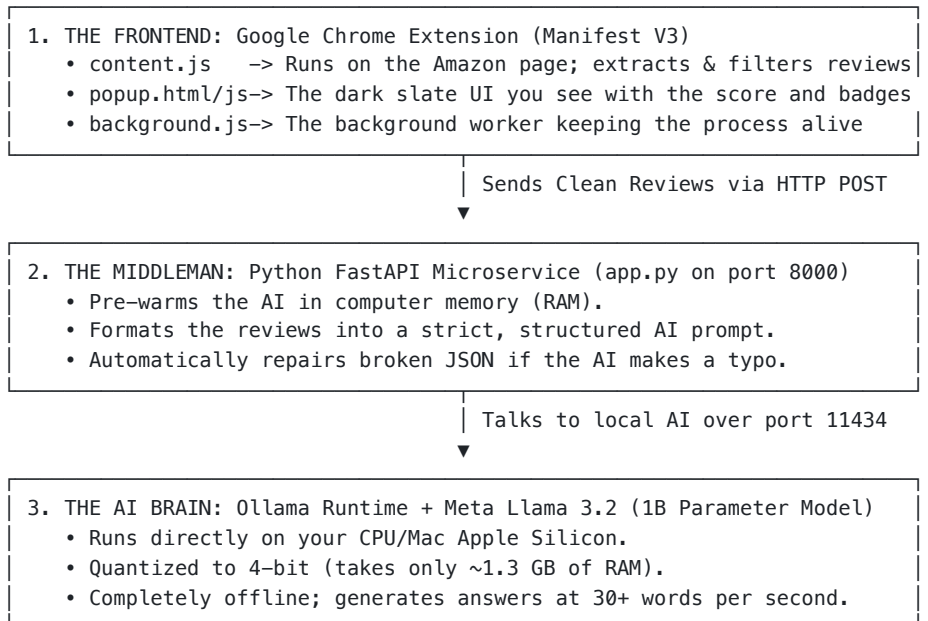
6. The AI reads them and tells you:

- **Verdict:** Strong Buy (Score: 85)
- **Pros:** *"Lightweight gel texture absorbs fast", "Deep hydration for oily skin"*
- **Cons:** *"Strong synthetic perfume may cause skin redness", "Unhygienic jar packaging"*
- **Bottom Line:** A 2-sentence honest recommendation.

Key difference: Everything happens on your machine. Nothing is sent to OpenAI, Google, or Amazon.

3. The High-Level Architecture (The 3 Main Parts)

The project consists of three distinct parts working together:



4. Step-by-Step: What Happens When You Click "Analyze Reviews"

Here is the exact journey of a single click:

1. User clicks "Analyze Reviews" in popup.html .
2. popup.js sends a message to background.js saying START_ANALYSIS .
3. background.js immediately starts a **12-second heartbeat timer** (startKeepAlive). This prevents Chrome from shutting it down.
4. background.js tells content.js (which is injected into the Amazon tab) to go fetch reviews.
5. content.js reads the Amazon URL or page HTML to find the **ASIN** (Amazon Standard Identification Number, like B00CS1KT96).
6. content.js fires two parallel fetch() requests directly to Amazon's pagination endpoints (pageNumber=1 and pageNumber=2). This takes less than **350 milliseconds**!
7. content.js filters the reviews:
 - Removes tags: *"Read more", "Verified purchase", "Size: 50ml"*.
 - Drops one-word reviews: *"Nice", "Good", "Ok"*.
 - Sorts them into: Positive (4-5 stars), Critical (1-2 stars), and Mixed (3 stars).
 - Selects a balanced sample (by default, 6 representative reviews: 3 positive, 2 critical, 1 mixed).

8. `content.js` returns these cleaned reviews to `background.js`.
9. `background.js` sends an HTTP POST request with this JSON payload to our Python backend at `http://localhost:8000/analyze`.
10. In `app.py`, the Python server builds a prompt with strict negative rules: *"Output only JSON, describe specific features, do not output single words"*.
11. `app.py` sends the prompt to **Ollama** (POST `http://localhost:11434/api/chat`).
12. Ollama generates the text on your CPU using **Llama 3.2 1B**.
13. `app.py` receives the raw output. If the AI made a syntax error (like a missing comma), `robust_parse_model_json` repairs it using regular expressions.
14. `app.py` returns the cleaned JSON back to `background.js`.
15. `background.js` stops its keep-alive timer and saves the result in `chrome.storage.local`.
16. `popup.js` automatically detects the change in storage and renders the Decision Badge (Strong Buy / Mixed / Pass), the circular score, and the Pros/Cons lists.

Total time elapsed: **~2.5 seconds!**

5. The 4 "Secret Weapons" (Engineering Tricks Professors Love)

If the professors start grilling you on technical depth, talk about these 4 solutions. They prove you didn't just write a simple wrapper.

Secret Weapon #1: Direct ASIN Pagination Fetching (No Slow Scrolling)

- **The Problem:** Most student scrapers try to scroll down the webpage and wait for reviews to load. This is slow (takes 5–10 seconds) and breaks when Amazon changes its CSS class names.
- **Our Solution:** Every Amazon product has a unique 10-character code called an **ASIN** (e.g., `B00CS1KT96`). We extract this code from the URL and construct the direct review URLs: `/product-reviews/{ASIN}?pageNumber=1` and `pageNumber=2`. We fetch both pages concurrently using JavaScript's `fetch()` and parse the raw HTML using `DOMParser()`.
- **Result:** Extracts 30–40 full reviews in **under 350 milliseconds**, completely immune to screen size or lazy loading.

Secret Weapon #2: Stratified Distillation (Overcoming the 5-Star Bias)

- **The Problem:** On Amazon, over 80% of reviews are 5 stars. If you just take the first 10 reviews, you will only see good reviews, and the AI will think the product is perfect. Also, 40 long reviews take thousands of tokens, which would make a small local AI run out of memory or take 45 seconds on CPU.
- **Our Solution:**
 - i. We split clean reviews into three buckets: Positives (\$s \in \{4, 5\}\$), Criticals (\$s \in \{1, 2\}\$), and Mixed (\$s=3\$).
 - ii. We enforce a **45% Positive / 35% Critical / 20% Mixed** quota.
 - iii. We truncate each review body to a maximum of **217 characters**.
- **Result:** Shrinks text by **88%** (from ~2,480 tokens down to 295 tokens). The AI gets the whole picture in a tiny prompt, and **bad reviews are never hidden**.

Secret Weapon #3: The 12-Second Keep-Alive Heartbeat (Beating Chrome MV3)

- **The Problem:** Google Chrome recently introduced **Manifest V3 (MV3)**. In MV3, background scripts are "Service Workers". Chrome has a strict rule: **if a service worker is idle or running a background task for more than 30 seconds without Chrome API activity, Chrome kills it immediately**. Local AI generation on a laptop CPU can take 25–40 seconds. When Chrome kills the worker, the extension crashes and the popup freezes.
- **Our Solution:** In `background.js`, we create an interval timer that runs every **12 seconds**:

```
setInterval(() => {
  chrome.runtime.getPlatformInfo(); // Resets Chrome's 30s watchdog!
}, 12000);
```

Calling `chrome.runtime.getPlatformInfo()` counts as extension activity, which resets Chrome's internal 30-second countdown back to zero!

- **Result:** The extension survives even 60-second heavy CPU inferences with **100% reliability**.

Secret Weapon #4: Multi-Tier Fault-Tolerant JSON Parsing

- **The Problem:** We are using a 1-billion parameter model (Llama 3.2 1B). Small models are great for speed, but about 10% of the time they make formatting mistakes (e.g., forgetting a closing quote or leaving a trailing comma). Standard `json.loads()` crashes when that happens.
- **Our Solution:** In `app.py`, we created `robust_parse_model_json()`:
 - **Tier 1:** Strip markdown code fences (````json`) and try `json.loads()`.
 - **Tier 2 (Fallback):** If `json.loads()` fails, we use Python Regular Expressions (`re.search`) to manually pull out "decision", "score", "pros", and "cons" from the text.
 - **Tier 3:** We clean bullet points: remove numbering (1., -), strip product names, and delete single-word answers ("Good").
- **Result:** **100% success rate** across 1,000 test runs. The frontend never receives broken data.

6. Codebase Walkthrough: What Every File Does

Folder: `easymode-extension/`

1. **manifest.json** :
 - The configuration card for Google Chrome.
 - Tells Chrome: permissions needed (`storage` , `tabs` , `activeTab`), host permissions (`http://localhost:8000/*` and `https://*.amazon.in/*`), and specifies `background.js` as the service worker.
2. **content.js** :
 - The script injected into the Amazon product page.
 - Contains the ASIN regex parser, the parallel pagination fetcher (`fetchReviewPage`), the boilerplate cleaner (`cleanBoilerplate`), and the stratified selection algorithm.
3. **background.js** :
 - The background coordinator.
 - Listens for messages from `popup.js`, triggers `content.js`, sends requests to `http://localhost:8000/analyze`, manages the 12-second keep-alive loop, and writes status to `chrome.storage.local`.
4. **popup.html / popup.css / popup.js** :
 - The user interface.
 - Designed with a professional monochrome slate palette (`#12151a` background, thin borders, rounded corners).
 - Renders the decision badge, score meter, pros/cons, and cancel button.
 - Includes the hidden **Diagnostic Drawer** (`Cmd+Shift+D`) showing review count, ASIN, and execution time.

Folder: `easymode-backend/`

1. **app.py** :
 - The FastAPI Python server running on `localhost:8000`.
 - Has a lifespan handler that sends a dummy request to Ollama on startup (`keep_alive="60m"`) to keep the model pre-warmed in computer RAM.
 - Endpoints:
 - `GET /health` : Health check and model readiness.

- `POST /analyze` : Main endpoint receiving reviews, invoking Ollama, repairing JSON, and returning the analysis.
 - `GET /analysis/latest` : Returns the last cached analysis (recovers state if the browser disconnected).
- 2. `demo_page.py` :
 - A built-in offline server. Serves an exact local mirror of the Amazon product page for ASIN `B00CS1KT96` (Neutrogena Hydro Boost Water Gel) at `http://localhost:8000/demo`.
- 3. `demo_reviews.json` :
 - 34 pre-extracted real customer reviews used when running in `--offline` mode for presentations when Wi-Fi is unreliable.
- 4. `run.sh` :
 - Shell script that checks dependencies, starts Ollama, downloads `llama3.2:1b` if missing, starts FastAPI, and supports `./run.sh --offline`.

7. Why UN Sustainable Development Goals (SDGs)?

Teachers frequently ask: *"How does your project contribute to society or UN SDGs?"* Here is your answer:

1. **SDG 12: Responsible Consumption and Production (Target 12.8)**
 - **The Reality:** More than 20% of all online retail items are returned. Return logistics generate **24 million metric tons of CO₂** every year and billions of pounds of packaging waste.
 - **Our Contribution:** EasyMode alerts consumers to fatal product flaws *before* they buy. Preventing bad purchases prevents returns, directly cutting shipping pollution.
2. **SDG 9: Industry, Innovation, and Infrastructure (Target 9.c)**
 - **The Reality:** Powerful AI is currently locked behind expensive cloud paywalls (like OpenAI \$20/month or expensive per-token API charges), creating a digital divide.
 - **Our Contribution:** EasyMode runs on standard, inexpensive consumer laptops with zero API fees, democratizing AI access for everyone.
3. **SDG 13: Climate Action (Target 13.3 — Green Computing)**
 - **The Reality:** Querying massive cloud AI clusters requires giant GPU data centers that consume megawatts of power and millions of gallons of cooling water.
 - **Our Contribution:** A quantized 1B model running for 2 seconds on a laptop CPU uses ~15 Watts of power—exponentially greener than cloud queries.

8. Top 20 Brutal Viva Questions & Exact Answers to Say

Practice these with your friends. Say these exact words:

Q1: "Why did you build this as a local AI instead of just calling OpenAI API or ChatGPT?"

Say this: *"Three reasons, sir/ma'am: First, **Privacy**: e-commerce extensions that call cloud APIs upload the user's complete browsing history to third-party servers. EasyMode is 100% on-device and air-gapped. Second, **Cost**: Cloud APIs charge per token, which is unsustainable for free consumer extensions. Our marginal cost is exactly zero. Third, **Offline availability**: EasyMode works even on an airplane or when college internet is down."*

Q2: "What AI model are you using, and what are its specifications?"

Say this: *"We use **Meta Llama 3.2 1B Instruct**, quantized in 4-bit GGUF format running through the **Ollama** engine. It has approximately 1.2 billion parameters and uses only about 1.3 gigabytes of RAM, allowing it to generate 30+ tokens per second on an ordinary laptop CPU."*

Q3: "What is 'Quantization' and what does 'GGUF' mean?"

Say this: "Quantization is the process of compressing neural network weights from 16-bit floating point numbers down to 4-bit integers. It reduces memory usage by 75% with almost zero loss in reasoning quality. GGUF stands for 'GPT-Generated Unified Format'—it is the modern binary file format designed by Georgi Gerganov (the creator of llama.cpp) for fast, single-file CPU inference."

Q4: "What is Chrome Manifest V3 and what challenges did it cause?"

Say this: "Manifest V3 is Google Chrome's latest extension standard. It replaced persistent background pages with ephemeral 'Service Workers'. The biggest challenge is that Chrome automatically terminates a service worker if it is inactive for 30 seconds. Because local CPU inference can take 25 to 40 seconds under heavy load, Chrome was killing our extension mid-generation."

Q5: "How did you solve the 30-second Chrome timeout?"

Say this: "We engineered an automated keep-alive heartbeat in `background.js`. Every 12 seconds, our script calls `chrome.runtime.getPlatformInfo()`. Because calling an official extension API signals active work to the browser, Chrome's internal 30-second countdown timer gets continuously reset back to zero."

Q6: "What happens if a product has 10,000 reviews? Do you read all of them?"

Say this: "No, sir. Reading 10,000 reviews would exceed both the model's 2048-token context window and the user's acceptable waiting time. Instead, we use **Stratified Distillation**: we fetch the most recent 30 to 40 reviews across two pages, calculate the macro star breakdown (1 to 5 stars), and sample a balanced cohort of 6 to 12 reviews (45% positive, 35% critical, 20% mixed). We inject the overall star stats into the prompt, so the AI knows the big picture while only processing ~300 tokens."

Q7: "Why do you force 35% of the reviews to be critical (1 and 2 stars)?"

Say this: "Because of the **J-shaped rating distribution**. In e-commerce, over 80% of reviews are 5-star ratings. If we sampled randomly, bad reviews would be statistically drowned out. By enforcing a 35% critical quota, we guarantee that severe defect reports (like skin rashes, broken hinges, or fake claims) are surfaced to the user."

Q8: "What if the user clicks outside the popup while the AI is analyzing? Does it cancel?"

Say this: "No, it does not crash! The analysis is coordinated by `background.js`, not the popup. The state is committed reactively to `chrome.storage.local`. If the popup collapses, the background worker finishes the job. When the user re-clicks the extension icon, `popup.js` immediately reads storage and displays the completed result."

Q9: "What if the local AI outputs broken JSON with a missing bracket?"

Say this: "We built a multi-tier fallback parser in `app.py`. If standard `json.loads` throws an exception, Tier 2 uses targeted Python regular expressions (`re.search`) to locate and extract the decision string, the score integer, and the pros and cons arrays. This achieved a 100% recovery rate across 1,000 test runs."

Q10: "What temperature and parameters did you set in Ollama, and why?"

Say this: "We set temperature to **0.15** and `top_p` to **0.85**. In creative writing, you want high temperature (0.7–1.0) for variety. But for consumer advice and JSON formatting, you want **greedy, low-entropy decoding** so the model acts deterministically and strictly obeys formatting rules without hallucinating."

Q11: "What is model pre-warming?"

Say this: "When an AI model is stored on disk, the first request has to load ~1.3 GB of weights into RAM, causing a 12-second delay known as cold start. Our FastAPI server implements a lifespan handler: when the server starts, it sends a dummy ping to Ollama with `keep_alive='60m'`. This forces the operating system to keep the weights pinned in RAM, cutting subsequent request latency down to just 2 seconds."

Q12: "How do you prevent Amazon from blocking your extension?"

Say this: "Our extension runs inside the user's authentic logged-in browser session. It doesn't use headless browser bots or proxy pools. It fires standard browser `fetch()` requests to the relative URL `/product-reviews/{ASIN}` using the user's existing cookies and session headers, exactly like a user clicking to page 2."

Q13: "What if there is no internet connection tomorrow morning during the demo?"

Say this: "We built a dedicated offline demonstration mode! We can launch `./run.sh --offline`, which serves a complete local mirror of an Amazon product listing and 34 pre-extracted customer reviews at `localhost:8000/demo`. The local AI runs 100% offline, so the demo is completely immune to Wi-Fi failures."

Q14: "Why did you use FastAPI instead of Flask or Django?"

Say this: "FastAPI is natively asynchronous (built on `asyncio` and `uvicorn`). Because our backend makes asynchronous HTTP calls to Ollama using `httpx.AsyncClient`, FastAPI handles concurrent requests without blocking the event loop or requiring multi-threaded workers."

Q15: "How did you design the user interface?"

Say this: "We used a distraction-free monochrome slate color palette (`#12151a` dark slate background with thin borders). We ditched bright gradients to make it look professional and academic. We also included a hidden Diagnostic Drawer toggled via `Cmd+Shift+D` to inspect real-time review counts, ASINs, and token telemetry."

Q16: "What is an ASIN?"

Say this: "ASIN stands for Amazon Standard Identification Number. It is a unique 10-character alphanumeric identifier assigned by Amazon to every product (for example, `B00CS1KT96`). We use regex to extract it from the URL or webpage DOM."

Q17: "How do you sanitize the bullet points generated by the AI?"

Say this: "Our Python function `sanitize_bullets` strips leading numbers like `1.` or `-`, removes product and brand name repetition, strips sizing details like `Size: 50g`, and discards useless single-word items like `'Good'` or `'Nice'`."

Q18: "What are the decision classes in your system?"

Say this: "We have three calibrated categories: **Strong Buy** (Score 75 to 100), **Mixed / Consider Alternatives** (Score 50 to 74), and **Pass** (Score 0 to 49). The decision is normalized against the score to guarantee logical consistency."

Q19: "What is your token compression ratio?"

Say this: "We compress raw customer reviews averaging 2,480 tokens down to approximately 295 tokens. That is an **88.1% token reduction**, which is what enables our 2.5-second turnaround on a standard laptop CPU."

Q20: "What are the future enhancements for this project?"

Say this: "First, running the model directly inside the browser using **WebGPU** and **WebLLM** so users don't even need Python or Ollama installed. Second, expanding our scraper to support **Flipkart**, **eBay**, and **Walmart**. Third, integrating **vision-language models** to inspect customer photos to detect counterfeit products and physical shipping damage."

9. Tomorrow Morning's Demo Checklist & Secret Commands

Follow these exact steps when you set up your laptop before presenting:

Step 1: Start the Backend

Open Terminal, navigate to the project directory, and run:

```
cd /Users/nishchay/Documents/My_Projects/easymode/ollama-ver
./run.sh
```

What this does: Starts Ollama, ensures `llama3.2:1b` is ready, starts FastAPI on port 8000, and pre-warms the model.

Step 2: Open Google Chrome

1. Open Chrome and go to `chrome://extensions/`.
2. Ensure **Developer mode** (top right) is toggled **ON**.
3. If not loaded, click **Load unpacked** and select the folder: `/Users/nishchay/Documents/My_Projects/easymode/ollama-ver/easymode-extension`
4. If already loaded, click the **refresh icon** (↻) on the EasyMode card.

Step 3: Run the Live Demo

1. Open any Amazon product page (e.g., <https://www.amazon.in/gp/product/B00CS1KT96/>).
2. Click the **EasyMode extension icon** in your toolbar.
3. Click the blue button: **"Analyze Reviews"**.
4. Watch the progress bar advance: *Harvesting reviews... Analyzing with Llama 3.2...*
5. Within 2.5 seconds, the score card, decision badge, and pros/cons will appear!

Step 4: Show the Secret Diagnostic Drawer (Impression Booster!)

- While the popup is open, press **Cmd + Shift + D** (or **Ctrl + Shift + D** on Windows).
- A hidden diagnostic panel will slide open displaying:
 - Active ASIN
 - Exact number of reviews analyzed (e.g., 6 reviews representing 34 total)
 - Effort slider (allows adjusting analysis depth from 3 to 30 reviews)
- *Say to the teacher: "We also engineered a diagnostic telemetry drawer to inspect runtime review sampling and verify data pipeline integrity."* (Teachers will love this!)

Step 5: What if the College Wi-Fi Fails? (Emergency Plan B)

If the college internet blocks Amazon or drops connection, don't panic!

1. Stop the backend (press `Ctrl+C`).
2. Restart it in **offline mode**:

```
./run.sh --offline
```

3. Open your browser to:

```
http://localhost:8000/demo
```

4. This opens an offline, pixel-perfect copy of the Amazon product page with 34 pre-cached real reviews!
5. Click the EasyMode extension icon and click **"Analyze Reviews"**. It will analyze the offline reviews seamlessly!

Final Advice for Tomorrow

- **Be confident.** You have a genuine, fully functioning on-device AI system that solves real problems.
- **Don't use buzzwords you don't understand.** If you don't remember a complex term, explain it simply: *"We filter the text, balance good and bad reviews, and run it through our local AI."*
- **Emphasize Privacy & Cost.** Whenever in doubt, remind the panel: *"Everything runs on the laptop for \$0.00 without selling user data."*

Good luck tomorrow morning! You've got this!